

Lecture 16 - Fisher's Linear Discriminant Analysis (FLDA)

(Parametric) Linear Models for Classification

So far we designed classifiers based on probability density or probability functions. In some cases, we saw that the resulting classifiers were equivalent to a set of linear discriminant functions.

We will now focus on the design of linear classifiers, irrespective of the underlying distributions describing the training data.

- The major advantage of linear classifiers is their simplicity and computational attractiveness.
- We will develop techniques for the computation of the corresponding linear functions. In the sequel we will focus on a more general problem, in which a linear classifier cannot classify correctly all feature vectors, yet we will seek ways to design an optimal linear classifier by adopting an appropriate optimality criterion.

Before, we designed generative classification algorithm that parametrize each class with a probabilistic model for the posterior probability of each class, i.e.,

$$P(C_k|x) = \frac{P(x|C_k)P(C_k)}{P(x)}.$$

Let's now consider a generalization of this model in which we transform the linear function of $\mathbf{w}$ using a nonlinear function $f(\bullet)$ so that:

$$y(\mathbf{x}) = f(\mathbf{w}^T \mathbf{x} + b)$$

In the machine learning literature $f(\bullet)$ is known as an **activation function**. The **decision surfaces** correspond to $y(\mathbf{x}) = \text{constant}$, so that $\mathbf{w}^T \mathbf{x} + b = \text{constant}$ and hence the decision surfaces are linear functions of $\mathbf{x}$, even if the function $f(\bullet)$ is nonlinear. For this reason, the class of models described by equation above are called **generalized linear models**.

Note, however, that in contrast to the models used for regression, they are no longer linear in the parameters due to the presence of the nonlinear function $f(\bullet)$. This will lead to more complex analytical and computational properties than for linear regression models. Nevertheless, these models are still relatively simple compared to the more general nonlinear models that will be studied later in the course.

The algorithm we will study will be equally applicable if we first make a fixed nonlinear transformation of the input variables using a vector of basis functions $\phi(x)$ as we did for regression models. In what follows, we consider classification directly in the original input space $\mathbf{x}$ but this illustration can be generalized to a notation involving basis functions.

Linear Discriminant Functions

A discriminant is a function that takes an input vector x and assigns it to one of K classes, denoted C_k .

Let's restrict our attention to **linear discriminants**, namely those for which the decision surfaces are hyperplanes. To simplify the discussion, we consider first the case of two classes and then investigate the extension to $K > 2$ classes.

Two Classes

The simplest representation of a linear discriminant function is obtained by taking a linear function of the input vector so that

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

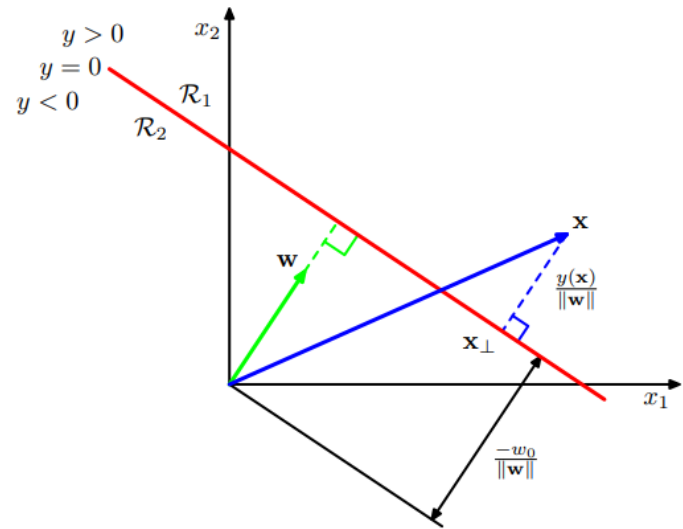
where $\mathbf{w}$ is called a **weight vector**, and b is a **bias** (not to be confused with bias in the statistical sense). The negative of the bias is sometimes called a threshold. An input vector $\mathbf{x}$ is assigned to class C_1 if $y(\mathbf{x}) \geq 0$ and to class C_2 otherwise.

Looks pretty familiar, right? If you are on one side of the line, then you are in class 1. If you are on the other side of the line, then you are in class 2. So, the decision boundary is $y(\mathbf{x}) = 0$.

- The vector $\mathbf{w}$ is orthogonal to every vector lying within the decision surface, and so $\mathbf{w}$ determines the orientation of the decision surface.
- Similarly, if $\mathbf{x}$ is a point on the decision surface, then $y(\mathbf{x}) = 0$, and so the normal distance from the origin to the decision surface is given by: $\frac{y(\vec{\mathbf{x}})}{\|\vec{\mathbf{w}}\|}$

```
In [7]: from IPython.display import Image
Image('figures/Figure4.1.png', width=800)
# Source: Bishop textbook
```

Out [7]: **Figure 4.1** Illustration of the geometry of a linear discriminant function in two dimensions. The decision surface, shown in red, is perpendicular to $\mathbf{w}$, and its displacement from the origin is controlled by the bias parameter w_0 . Also, the signed orthogonal distance of a general point $\mathbf{x}$ from the decision surface is given by $y(\mathbf{x})/\|\mathbf{w}\|$.



For a 2-class problem, we use the following **decision rule**:

$$\text{Decide } \begin{cases} \mathbf{x} \in C_1 & \text{if } y(\mathbf{x}) > 0 \\ \mathbf{x} \in C_2 & \text{if } y(\mathbf{x}) < 0 \end{cases}$$

We already saw an approach of solving for this mapper function, specifically the Least Squares solution for regression tasks.

We can use the LS solution for classification tasks (take class 1 if $y(x) > 0$ or 0 otherwise). But, the Gaussianity assumptions (objective function modeled as the MSE function) no longer applies. Moreover, since the standard LS uses the MSE objective function, its goal is no minimize the error and will, thus, become very sensitive to points that are "outliers" but yet correctly classified (as seen in picture below).

```
In [8]: Image('figures/Figure4.4.png', width=800)
# Source: Bishop textbook
```

Out [8]:

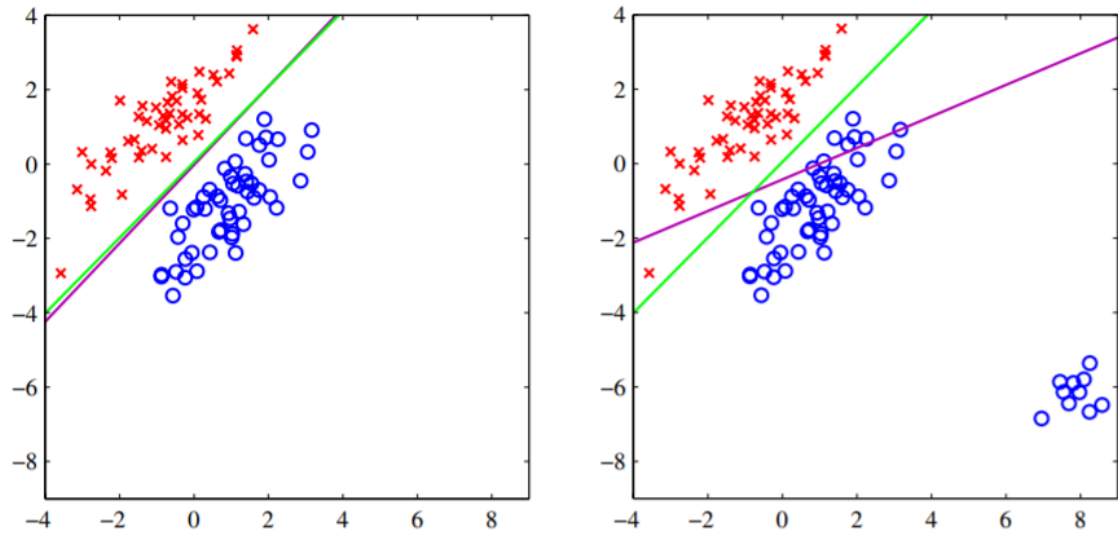


Figure 4.4 The left plot shows data from two classes, denoted by red crosses and blue circles, together with the decision boundary found by least squares (magenta curve) and also by the logistic regression model (green curve), which is discussed later in Section 4.3.2. The right-hand plot shows the corresponding results obtained when extra data points are added at the bottom left of the diagram, showing that least squares is highly sensitive to outliers, unlike logistic regression.

We will study 4 other methods to optimize the parameter $\mathbf{w}$ of a linear discriminant function (classifier), namely:

1. Fisher's Linear Discriminant Analysis (FLDA or LDA)
2. The Perceptron algorithm
3. Logistic Regression
4. Support Vector Machine (SVM)

Fisher's Linear Discriminant Analysis (or LDA)

A very popular type of a linear discriminant is the **Fisher's Linear Discriminant**.

- Given two classes, we can compute the mean of each class:

$$\vec{\mathbf{m}}_1 = \frac{1}{N_1} \sum_{n \in C_1} \vec{\mathbf{x}}_n$$

$$\vec{\mathbf{m}}_2 = \frac{1}{N_2} \sum_{n \in C_2} \vec{\mathbf{x}}_n$$

We can maximize the separation of the means:

$$m_2 - m_1 = \vec{\mathbf{w}}^T (\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)$$

- $\vec{\mathbf{w}}^T \vec{\mathbf{x}}$ takes a D dimensional data point and projects it down to 1-D with a weight sum of the original features. We want to find a weighting that maximizes the separation of the class means.
- Not only do we want well separated means for each class, but we also want each class to be *compact* to minimize overlap between the classes.
- Consider the *within class variance*:

$$\begin{aligned} s_k^2 &= \sum_{n \in C_k} (y_n - m_k)^2 = \sum_{n \in C_k} (\vec{\mathbf{w}}^T \vec{\mathbf{x}}_n - m_k)^2 \\ &= \vec{\mathbf{w}}^T \sum_{n \in C_k} (\vec{\mathbf{x}}_n - \vec{\mathbf{m}}_k)(\vec{\mathbf{x}}_n - \vec{\mathbf{m}}_k)^T \vec{\mathbf{w}} \end{aligned}$$

- So, we want to minimize within class variance and maximize between class separability. How about the following objective function:

$$\begin{aligned} J(\mathbf{w}) &= \frac{(m_2 - m_1)^2}{s_1^2 + s_2^2} \\ &= \frac{\vec{\mathbf{w}}^T (\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)(\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)^T \vec{\mathbf{w}}}{\sum_{n \in C_1} (\vec{\mathbf{w}}^T \vec{\mathbf{x}}_n - m_1)^2 + \sum_{n \in C_2} (\vec{\mathbf{w}}^T \vec{\mathbf{x}}_n - m_2)^2} \\ &= \frac{\vec{\mathbf{w}}^T (\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)(\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)^T \vec{\mathbf{w}}}{\vec{\mathbf{w}}^T \left(\sum_{n \in C_1} (\vec{\mathbf{x}}_n - \vec{\mathbf{m}}_1)(\vec{\mathbf{x}}_n - \vec{\mathbf{m}}_1)^T + \sum_{n \in C_2} (\vec{\mathbf{x}}_n - \vec{\mathbf{m}}_2)(\vec{\mathbf{x}}_n - \vec{\mathbf{m}}_2)^T \right) \vec{\mathbf{w}}} \\ &= \frac{\vec{\mathbf{w}}^T \mathbf{S}_B \vec{\mathbf{w}}}{\vec{\mathbf{w}}^T \mathbf{S}_W \vec{\mathbf{w}}} \end{aligned}$$

where

$$\mathbf{S}_B = (\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)(\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)^T$$

and

$$\mathbf{S}_W = \sum_{n \in C_1} (\vec{\mathbf{x}}_n - \vec{\mathbf{m}}_1)(\vec{\mathbf{x}}_n - \vec{\mathbf{m}}_1)^T + \sum_{n \in C_2} (\vec{\mathbf{x}}_n - \vec{\mathbf{m}}_2)(\vec{\mathbf{x}}_n - \vec{\mathbf{m}}_2)^T$$

- Ok, so let's optimize:

$$\begin{aligned}
\frac{\partial J(\vec{\mathbf{w}})}{\partial \vec{\mathbf{w}}} &= \frac{2(\vec{\mathbf{w}}^T \mathbf{S}_W \vec{\mathbf{w}}) \mathbf{S}_B \vec{\mathbf{w}} - 2(\vec{\mathbf{w}}^T \mathbf{S}_B \vec{\mathbf{w}}) \mathbf{S}_W \vec{\mathbf{w}}}{(\vec{\mathbf{w}}^T \mathbf{S}_W \vec{\mathbf{w}})^2} = 0 \\
0 &= \frac{\mathbf{S}_B \vec{\mathbf{w}}}{(\vec{\mathbf{w}}^T \mathbf{S}_W \vec{\mathbf{w}})} - \frac{(\vec{\mathbf{w}}^T \mathbf{S}_B \vec{\mathbf{w}}) \mathbf{S}_W \vec{\mathbf{w}}}{(\vec{\mathbf{w}}^T \mathbf{S}_W \vec{\mathbf{w}})^2} \\
(\vec{\mathbf{w}}^T \mathbf{S}_W \vec{\mathbf{w}}) \mathbf{S}_B \vec{\mathbf{w}} &= (\vec{\mathbf{w}}^T \mathbf{S}_B \vec{\mathbf{w}}) \mathbf{S}_W \vec{\mathbf{w}} \\
\mathbf{S}_B \vec{\mathbf{w}} &= \frac{\vec{\mathbf{w}}^T \mathbf{S}_B \vec{\mathbf{w}}}{\vec{\mathbf{w}}^T \mathbf{S}_W \vec{\mathbf{w}}} \mathbf{S}_W \vec{\mathbf{w}} \\
\mathbf{S}_W^{-1} \mathbf{S}_B \vec{\mathbf{w}} &= \lambda \vec{\mathbf{w}}
\end{aligned}$$

where the scalar $\lambda = \frac{\vec{\mathbf{w}}^T \mathbf{S}_B \vec{\mathbf{w}}}{\vec{\mathbf{w}}^T \mathbf{S}_W \vec{\mathbf{w}}}$

Does this look familiar?

This is the generalized eigenvalue problem!

- So the direction of projection correspond to the eigenvectors of $\mathbf{S}_W^{-1} \mathbf{S}_B$ with the largest eigenvalues.

The solution is easy when $S_w^{-1} = (\Sigma_1 + \Sigma_2)^{-1}$ exists.

In this case, if we use the definition of $S_B = (\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)(\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)^T$:

$$\begin{aligned}
S_W^{-1} S_B \vec{\mathbf{w}} &= \lambda \vec{\mathbf{w}} \\
S_W^{-1} (\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1) (\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)^T \vec{\mathbf{w}} &= \lambda \vec{\mathbf{w}}
\end{aligned}$$

Noting that $\alpha = (\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)^T \vec{\mathbf{w}}$ is a constant, this can be written as:

$$S_W^{-1} (\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1) = \frac{\lambda}{\alpha} \vec{\mathbf{w}}$$

- Since we don't care about the magnitude of $\vec{\mathbf{w}}$:

$$\vec{\mathbf{w}}^* = S_W^{-1} (\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1) = (\Sigma_1 + \Sigma_2)^{-1} (\vec{\mathbf{m}}_2 - \vec{\mathbf{m}}_1)$$

Make sure $\vec{\mathbf{w}}^*$ is a unit vector by taking: $\vec{\mathbf{w}}^* \leftarrow \frac{\vec{\mathbf{w}}^*}{\|\vec{\mathbf{w}}^*\|}$

- Note that if the within-class covariance, S_W , is isotropic, so that S_W is proportional to the unit matrix, we find that $\vec{\mathbf{w}}$ is proportional to the difference of the class means.

- This result is known as *Fisher's linear discriminant*, although strictly it is not a discriminant but rather a specific choice of direction for projection of the data down to one dimension. However, the projected data can subsequently be used to construct a discriminant, by choosing a threshold y_0 so that we classify a new point as belonging to C_1 if $y(x) \geq y_0$ and classify it as belonging to C_2 otherwise.

Also, note that:

- For a classification problem with Gaussian classes of equal covariance $\Sigma_i = \Sigma$, the boundary is the plane of normal:

$$\vec{w} = \Sigma^{-1}(\vec{m}_i - \vec{m}_j)$$

- If $\Sigma_2 = \Sigma_1$, this is also the LDA solution.

This gives two different **interpretations** of LDA:

- It is optimal *if and only if* the classes are Gaussian and have equal covariance.
- A classifier on the LDA features, is equivalent to the boundary after the approximation of the data by two Gaussians with equal covariance.

The final discriminant decision boundary is $\vec{y} = \vec{w}^* \vec{x} + w_0$

The *bias* term w_0 can be defined as:

$$w_0 = -\frac{1}{2} \left(\frac{1}{N_1} \sum_{n \in C_1} \vec{x}_n + \frac{1}{N_2} \sum_{n \in C_2} \vec{x}_n \right)^T \vec{w}^*$$

- An extension to multi-class problems has a similar derivation.

Limitations of LDA:

1. LDA produces at most $C - 1$ feature projections, where C is the number of classes.
2. If the classification error estimates establish that more features are needed, some other method must be employed to provide those additional features.
3. LDA is a parametric method (it assumes unimodal Gaussian likelihoods).
4. If the distributions are significantly non-Gaussian, the LDA projections may not preserve complex structure in the data needed for classification.
5. LDA will also fail if discriminatory information is not in the mean but in the variance of the data.

A popular variant of LDA are the **Multi-Layer Perceptrons** (or MLPs).
